ResumeIQ

Smart Resume Analyzer

Beginner's Project Learning Guide

A complete, beginner-friendly explanation of the ResumeIQ project: architecture, code, algorithms, setup, testing, and college viva preparation. Built with Python and Streamlit. Rule-based and offline.

All scores described here are rule-based educational indicators - not a prediction of hiring success and not a guarantee of ATS compatibility.

Table of Contents

Chapter 1 - Project Overview **4**
1.1 What is this project? (in one simple sentence) 4
1.2 Important words, explained before we use them 4
1.3 The real-world problem it solves 4
1.4 Who uses it? 4
1.5 What it does NOT do (be honest in your viva!) 4
1.6 The technology used (and why) 5
1.7 The files in the project 5
1.8 The three-layer architecture (text diagram) 6
1.9 A realistic example (what actually happens) 7
1.10 Checklist for this chapter 7
Chapter 2 - Python & Technology Basics (Prerequisites) **8**
2.1 Modules and imports 8
2.2 Functions - parameters, return values, and defaults 8
2.3 Strings and string methods 9
2.4 Lists and list comprehensions 9
2.5 Dictionaries and the "dictionary of lists" 10
2.6 Sets and set operations * (the heart of the scoring) 10
2.7 f-strings (formatted strings) 11
2.8 Truthiness (empty things are "False") 11
2.9 Regular expressions (the re module) * 11
2.10 Exception handling (try / except) 12
2.11 Files and bytes (reading an uploaded file) 12
2.12 Classes and objects (just enough for the tests) 13
2.13 Streamlit (turning Python into a web page) 13
2.14 pandas (tables of data) 14
2.15 Plotly (charts) 14
2.16 Virtual environments & pip (setting up) 14
2.17 Checklist for this chapter 15
Chapter 3 - File-by-File Code Explanation **16**
3.1 sample_data.py - the data layer 16
3.2 analyzer.py - the engine (the most important file) 17
3.3 app.py - the web page (the UI layer) 22
3.4 tests/test_analyzer.py - the tests 23
3.5 Files-explained checklist 24
Chapter 4 - Execution Flow & Algorithms **25**
4.1 How the program starts (startup sequence) 25
4.2 What happens when you click "Analyze" (end-to-end trace) 25
4.3 What each major button does 26
4.4 Algorithm 1 - Text preprocessing 26

4.5 Algorithm 2 - Safe skill matching (the regex trick) 27
4.6 Algorithm 3 - Keyword match score (set maths) * 27
4.7 Algorithm 4 - Readiness score (weighted blend) 28
4.8 Putting it together - a mini sequence diagram 28
4.9 Checklist for this chapter 29

Chapter 5 - Installation, Testing & Troubleshooting 30

5.1 Prerequisites 30
5.2 Step-by-step setup 30
5.3 What you should see when it works 30
5.4 Configuration & environment variables 31
5.5 Trying the main feature (manual test) 31
5.6 Running the automated tests 31
5.7 Stopping the app 32
5.8 Troubleshooting (common errors & fixes) 32
5.9 How to debug (for a beginner) 32
5.10 Edge cases that matter (and are handled) 32
5.11 Known limitations (be honest in the viva) 33
5.12 Checklist for this chapter 33

Chapter 6 - College Viva Preparation 34

6.1 Explain the project in three lengths 34
6.2 Demonstration script (step-by-step) 35
6.3 Fifty+ viva questions with answers 35
6.4 Code-specific questions (based on the real files) 37
6.5 Explain it to different audiences 38

Chapter 7 - Glossary & Revision Notes 39

7.1 Glossary (plain definitions + project example) 39
7.2 One-page revision sheet 39
7.3 The 20 most important things to remember 40
7.4 Important files and their responsibilities 40
7.5 Most important functions and their purpose 41
7.6 Execution flow in one line 41
7.7 Common viva mistakes to avoid 41
7.8 Final self-assessment checklist 41

Chapter 1 - Project Overview

*How to read this chapter: Start here. It explains *what* the project is, *why* it exists, and *how its pieces fit together*, all in plain English. You do not need to understand any code yet.*

1.1 What is this project? (in one simple sentence)

ResumeIQ is a small website (that runs on your own computer) which reads a resume and a job description, then tells you how well they match by looking for important skills (like *Python*, *SQL*, *Excel*) in both.

That is the whole idea. Everything else in the code is just doing that job carefully and showing the result nicely.

1.2 Important words, explained before we use them

Word	Simple meaning
Resume	A document describing a person's skills, education and experience.
Job description (JD)	A document from a company listing what skills a job needs.
Skill / keyword	An important word we look for, e.g. python, sql, aws.
Match	A skill that appears in both the resume and the job description.
Score	A number (0 to 100) showing how many required skills the resume has.
Streamlit	A free Python tool that turns a Python script into a web page.
Rule-based	The program follows simple, fixed rules we wrote - it does not use AI.
Embedded strings	Example text (sample resumes, jobs, skills) written directly inside the code.

*(!) "Embedded strings" does NOT mean AI "embeddings". In this project it simply means *text typed directly into the Python files* using triple quotes ("""" ... """"). There is no machine learning anywhere in this project.*

1.3 The real-world problem it solves

- For a student / job seeker: "Does my resume contain the skills this job is asking for? What am I missing?"
- For a recruiter: "Out of many resumes, which ones mention the required skills?" (a quick first filter).
- For a teacher / examiner: a clear, honest example of Python text processing that a student can fully explain.

Doing this by hand is slow and easy to get wrong. ResumeIQ does it in seconds and shows exactly why it gave each number.

1.4 Who uses it?

Students, job seekers, recruiters (for a first pass only), and faculty evaluating the mini-project. The web page is simple enough for someone with no programming knowledge to use.

1.5 What it does NOT do (be honest in your viva!)

The code itself is honest about its limits, and you should be too:

- It matches words, it does not understand meaning. If a resume says *"built ML models"* but never writes *"machine learning"*, the tool may miss it. (This is called a false negative.)
- It cannot judge how *good* a candidate is, or their real experience level.
- It cannot guarantee it will pass a company's real ATS (Applicant Tracking System).
- It never guesses personal things like age, gender or religion.

This honesty is written directly in the code and on the "About & Limitations" page of the app.

1.6 The technology used (and why)

Technology	What it is	Why this project uses it
Python 3	The programming language.	Easy to read; great at text handling.
Streamlit	Turns a Python file into a web app.	Build a web page with charts using only Python - no HTML/JavaScript.
pandas	Works with tables of data.	To show results as neat tables and make CSV files.
Plotly	Draws interactive charts.	For the gauges and bar charts.
pypdf	Reads text out of PDF files.	So users can upload a .pdf resume.
python-docx	Reads text out of Word files.	So users can upload a .docx resume.
fpdf2	Creates PDF files.	To let users download a PDF report.
re (built-in)	Python's "regular expressions".	To find emails, phones and skills by pattern.
unittest (built-in)	Python's testing tool.	To automatically check the logic is correct.

Built-in vs installed: re and unittest come *with* Python - you do not install them. The others (Streamlit, pandas, Plotly, pypdf, python-docx, fpdf2) are external packages you install with pip (see Chapter 5).

1.7 The files in the project

This is the real folder layout (ignoring auto-generated caches and the venv/ virtual environment folder):

```
Resume Analyzer/
|
+-- app.py                <- The web page: buttons, charts, uploads (Streamlit)
+-- analyzer.py           <- The "brain": all the analysis logic
+-- sample_data.py        <- Embedded samples: resumes, jobs, and the skill list
+-- requirements.txt      <- The list of packages to install
+-- README.md             <- Beginner install & run guide
+-- PROJECT_REPORT.md     <- The formal academic report
+-- VIVA_GUIDE.md         <- A short viva guide (this learning guide is bigger)
+-- .gitignore            <- Tells Git which files to ignore
|
+-- tests/
|   +-- test_analyzer.py   <- 31 automatic tests for analyzer.py
|
+-- project_learning_guide/  <- (NEW) this beginner learning guide
    +-- README.md
    +-- 01_Project_Overview.md
```

```
    +-- 02_Python_and_Technology_Basics.md
    +-- 03_File_by_File_Code_Explanation.md
    +-- 04_Execution_Flow_and_Algorithms.md
    +-- 05_Installation_Testing_and_Troubleshooting.md
    +-- 06_College_Viva_Preparation.md
    +-- 07_Glossary_and_Revision_Notes.md
    +-- Complete_Project_Explanation.pdf
```

What each important file is responsible for

File	Responsibility
sample_data.py	Stores the data: SKILL_DATABASE (skills by category), 5 sample resumes, 5 sample job descriptions. It has no logic - just data.
analyzer.py	The engine. Cleans text, finds skills, compares them with set maths, calculates scores, writes reports. It knows nothing about the web page, so it can be tested on its own.
app.py	The face of the app. Draws the pages, takes user input, calls analyzer.py, and shows the results as metrics, tables and charts.
tests/test_analyzer.py	Checks that analyzer.py behaves correctly for many situations (empty inputs, tricky skills like c++, etc.).

1.8 The three-layer architecture (text diagram)

The project is built in three layers. Think of a restaurant:
the waiter (UI) takes your order, the kitchen (logic) cooks it, and the pantry (data) stores the ingredients.

```
         +-----------------------------------------------+
         |                  app.py                       |
 USER  ->  |   (Streamlit UI = the "waiter")               |
 (you) <-  |   pages, buttons, file upload, charts,        |
         |   download report / CSV                       |
         +---------------+-------------------------------+
                         | calls functions in
                         v
         +-----------------------------------------------+
         |                analyzer.py                    |
         |   (the "kitchen" = all the thinking)          |
         |   preprocess_text(), extract_skills(),        |
         |   calculate_keyword_match(),                  |
         |   calculate_readiness_score(),                |
         |   analyze_resume() ...                        |
         +---------------+-------------------------------+
                         | reads data from
                         v
         +-----------------------------------------------+
         |               sample_data.py                  |
         |   (the "pantry" = the ingredients)            |
         |   SKILL_DATABASE, SAMPLE_RESUMES,             |
         |   SAMPLE_JOB_DESCRIPTIONS                     |
         +-----------------------------------------------+

         +-----------------------------------------------+
         |           tests/test_analyzer.py              |
         |   imports and checks analyzer.py directly     |
         |   (does NOT need the web page to run)         |
         +-----------------------------------------------+
```

Explaining each arrow

- USER -> app.py: You type/paste/upload a resume and job description and click a button.
- app.py -> analyzer.py: The web page hands the text to the engine by calling a function like analyze_resume(resume_text, job_text).
- analyzer.py -> sample_data.py: The engine reads the list of known skills (SKILL_DATABASE) to know what to look for.
- analyzer.py -> app.py: The engine returns one big dictionary of results.
- app.py -> USER: The page turns that dictionary into numbers, tables and charts you can see, plus downloadable files.
- tests -> analyzer.py: The tests call the engine directly to confirm it is correct - they never open the web page.

Why split into three layers? So each part is simple and can be understood, changed, and tested on its own. This is called separation of concerns, and it is one of the strongest things you can say about this project in your viva.

1.9 A realistic example (what actually happens)

1. You open the app and choose the "Python Developer" sample (resume + job).
2. The resume mentions: python, sql, flask, streamlit, pandas, git, docker...
3. The job asks for: python, sql, flask, streamlit, pandas, git, docker...
4. The engine finds the skills in both, compares them, and reports:
    - Keyword match: 100% (every required skill is present)
    - Missing skills: none
    - A Readiness score out of 100 and a downloadable PDF report.
5. If instead you test the Data Analyst resume against the Python Developer job, the score is lower, because that resume is missing skills like flask, streamlit, docker. This visible difference is great for a demo.

(These outcomes are produced by the real code; see Chapter 4 for the exact step-by-step trace.)

1.10 Checklist for this chapter

- [ ] I can say in one sentence what ResumeIQ does.
- [ ] I can explain "embedded strings" and why it is not AI.
- [ ] I can name the three layers (UI / logic / data) and the file for each.
- [ ] I can state two honest limitations of the tool.
- [ ] I can draw the three-layer diagram from memory.

Next: Chapter 2 teaches the Python ideas you need before reading the code.

Chapter 2 - Python & Technology Basics (Prerequisites)

How to read this chapter: These are the building blocks used in the project. Each topic has: a simple definition, a real-world analogy, a tiny example, a line-by-line explanation, where it appears in our project, and why the project needs it. Read them in order - each one builds on the last.

We assume you already know: variables, numbers/strings/booleans, if/else, for loops, functions (basic), lists, and dictionaries. Everything below is treated as new.

Learning order (easiest -> hardest):
1. Modules & imports -> 2. Functions (params, return, defaults) ->
3. Strings & string methods -> 4. Lists & comprehensions -> 5. Dictionaries (and dictionary-of-lists) -> 6. Sets & set operations -> 7. f-strings ->
8. Truthiness -> 9. Regular expressions -> 10. Exceptions (try/except) ->
11. Files & bytes -> 12. Classes (just enough for the tests) ->
13. Streamlit -> 14. pandas -> 15. Plotly -> 16. Virtual environments & pip.

2.1 Modules and imports

Definition: A module is just a Python file (e.g. analyzer.py).
Importing means "let me use the code written in another file".

Analogy: A module is a toolbox. import is you borrowing a specific tool from it instead of making your own.

Tiny example:

```
import re                       # borrow Python's regular-expression toolbox
from sample_data import SKILL_DATABASE   # borrow ONE thing from our own file
```

- Line 1: load the whole re module; use it as re.something().
- Line 2: reach into sample_data.py and take just the SKILL_DATABASE variable, so we can write SKILL_DATABASE directly.

Where in our project: top of analyzer.py (lines 34-37) and top of app.py (it imports analyzer and several items from sample_data).

Why needed: It lets the three layers (UI, logic, data) live in separate files but still work together.

2.2 Functions - parameters, return values, and defaults

Definition: A function is a named block of code that takes inputs (parameters), does something, and usually gives back an output (return). A default value lets a parameter be optional.

Analogy: A coffee machine. You put in inputs (water, beans), press start, and it returns coffee. A default is "if you don't choose a size, you get medium".

Tiny example:

```
def greet(name, greeting="Hello"):
    return greeting + ", " + name + "!"
```

```
print(greet("Arul"))            # Hello, Arul!
print(greet("Arul", "Welcome")) # Welcome, Arul!
```

- name is required; greeting="Hello" is optional (has a default).
- return hands the finished string back to whoever called greet.

Where in our project: everywhere. For example extract_skills(text, skill_database=SKILL_DATABASE) in analyzer.py (line 140) - skill_database has a default, so you can call extract_skills(text) and it automatically uses the project's skill list.

Why needed: Functions let us name each step (preprocess_text, calculate_keyword_match) and reuse it without copying code.

2.3 Strings and string methods

Definition: A string is text. A method is a function attached to a value, called with a dot: text.lower().

Analogy: If a string is a sentence written on paper, methods are actions you can do to it: make it lowercase, cut it into words, trim the edges.

The string methods this project uses the most:

```
s = "  Hello WORLD\nPython  "
s.lower()        # '  hello world\npython  '   (make lowercase)
s.strip()        # 'Hello WORLD\nPython'       (remove edge whitespace)
s.split()        # ['Hello', 'WORLD', 'Python'] (cut into words on whitespace)
s.splitlines()   # ['  Hello WORLD', 'Python  '] (cut into lines)
```

- .lower() -> used so "Python" and "python" are treated the same.
- .split() -> with no argument, splits on any whitespace and drops blanks; used to count words.
- .splitlines() -> breaks text into lines; used to count non-empty lines.

Where in our project: preprocess_text() (analyzer.py line 44) uses .lower(); count_resume_statistics() (line 195) uses .split() and .splitlines().

Why needed: The whole project is about processing text, so these are the core tools.

2.4 Lists and list comprehensions

Definition: A list is an ordered collection in square brackets: [1, 2, 3]. A list comprehension is a short way to build a new list with a loop inside brackets.

Analogy: A list is a shopping list. A comprehension is "write down every fruit from the basket that is ripe" - in one line.

Tiny example:

```
numbers = [1, 2, 3, 4]
evens = [n for n in numbers if n % 2 == 0]   # [2, 4]
```

Read it as: *"collect n, for each n in numbers, if n is even."*

Where in our project: count_resume_statistics() line 216:

```
line_count = len([ln for ln in text.splitlines() if ln.strip()])
```

This builds a list of only the non-empty lines, then len(...) counts them.

Why needed: Comprehensions make the code short and readable when filtering or transforming lists.

2.5 Dictionaries and the "dictionary of lists"

Definition: A dictionary stores key: value pairs in curly braces: {"name": "Arul"}. A dictionary of lists has lists as its values.

Analogy: A dictionary is a real dictionary - you look up a *word* (key) to get its *meaning* (value). A dictionary of lists is like a filing cabinet: each labelled drawer (key) holds many papers (a list).

Tiny example (exactly the shape our project uses):

```
skills = {
    "Programming Languages": ["python", "java", "c++"],
    "Databases": ["sql", "mysql"],
}
skills["Databases"]          # ['sql', 'mysql']
for category, items in skills.items():   # loop over key AND value together
    print(category, "has", len(items), "skills")
```

- skills["Databases"] looks up one drawer.
- .items() gives you both the key and the value each loop - used constantly.

Where in our project: SKILL_DATABASE in sample_data.py is a dictionary of lists (10 categories, each a list of skill words). extract_skills() loops over it with for category, skills in skill_database.items(): (line 160).

Why needed: It keeps skills organised by category, which is used for the category-coverage chart and the skill-gap table.

2.6 Sets and set operations * (the heart of the scoring)

Definition: A set is an unordered collection with no duplicates, written with curly braces: {1, 2, 3}. Sets support maths operations:
- intersection (& or .intersection()) -> items in both sets.
- difference (- or .difference()) -> items in the first but not the second.

Analogy: Two circles overlapping (a Venn diagram). The overlap is the intersection; the part of circle A outside B is the difference A - B.

Tiny example:

```
resume = {"python", "sql", "git"}
job    = {"python", "sql", "docker", "aws"}

matched = resume.intersection(job)   # {'python', 'sql'}   -> in BOTH
missing = job.difference(resume)     # {'docker', 'aws'}   -> job needs, resume lacks
extra   = resume.difference(job)     # {'git'}             -> bonus skills
```

Where in our project: calculate_keyword_match() (analyzer.py lines 392-394) - this is exactly how the match is computed:

```
matched = resume_skills.intersection(job_skills)
missing = job_skills.difference(resume_skills)
```

```
extra   = resume_skills.difference(job_skills)
```

Why needed: Sets make the comparison fast, duplicate-free, and easy to explain ("matched = overlap; missing = what the job has that the resume doesn't"). This is the single most important concept to master for the viva.

2.7 f-strings (formatted strings)

Definition: An f-string lets you drop variables straight into text by putting an f before the quotes and {variable} inside.

Tiny example:

```
count = 5
print(f"You matched {count} skills.")   # You matched 5 skills.
```

Where in our project: everywhere a message or report line is built, e.g. f"Matched {km['matched_count']} of {km['total_job_keywords']} ..." (analyzer.py line 688).

Why needed: Clean, readable way to build the report text and UI messages.

2.8 Truthiness (empty things are "False")

Definition: In an if, Python treats some values as False even though they are not literally False: empty string "", empty list [], 0, and None are all "falsy". Non-empty values are "truthy".

Tiny example:

```
text = ""
if not text:
    print("the text is empty")   # this runs
```

Where in our project: preprocess_text() line 62 if not text or not isinstance(text, str): return "" - safely handles empty or non-text input. Also ", ".join(km["missing"]) or "None" shows the word "None" when the list is empty.

Why needed: It guards against empty inputs so the app never crashes.

2.9 Regular expressions (the re module) *

Definition: A regular expression (regex) is a mini-language for describing patterns in text, so you can find things like "an email" or "a whole word". Python's re module runs them.

Analogy: A very smart "Find" box. Instead of searching for one exact word, you search for a *shape*, e.g. "some letters, then @, then more letters, then a dot".

The regex pieces used in this project:

Piece	Meaning	Example
\d	any digit 0-9	\d\d matches 42

Piece	Meaning	Example
\s	any whitespace	spaces, tabs, newlines
[...]	any one character from a set	[a-z] = one lowercase letter
[^...]	any character not in the set	[^a-z] = not a lowercase letter
+	"one or more of the previous"	\d+ = one or more digits
\b	a word boundary (edge of a word)	\bcat\b matches cat but not category
(?<!x)	negative look-behind: "not preceded by x"	used for c++
(?!x)	negative look-ahead: "not followed by x"	used for c++

Tiny example:

```
import re
re.findall(r"\d+", "I have 3 cats and 12 fish")   # ['3', '12']  (all digit runs)
re.sub(r"[^a-z ]", " ", "Hi, THERE!")             # 'hi there '   (depends on lowercase)
re.search(r"\bpython\b", "i love python")         # a match object (found!)
```

- findall -> return all matches as a list.
- sub -> substitute (replace) matches with something else.
- search -> find the first match (or None).

Where in our project:
- preprocess_text() uses re.sub to clean text (lines 71, 75).
- Contact detection uses compiled patterns for email/phone/LinkedIn (analyzer.py lines 246-250).
- Skill matching uses \b for normal skills and look-arounds for c++/c# (_build_skill_pattern, lines 106-129).

Why needed: Regex is how the project finds emails, phone numbers, and whole-word skills safely (so r doesn't match inside react).

2.10 Exception handling (try / except)

Definition: Code can fail (e.g. a broken file). try runs risky code; except catches the error so the program keeps going instead of crashing. raise deliberately creates an error with a clear message.

Analogy: A safety net under a trapeze artist. If they fall (error), the net catches them (except) instead of disaster (crash).

Tiny example:

```
try:
    number = int("hello")      # this fails
except ValueError:
    number = 0                 # we handle it calmly
print(number)                  # 0
```

Where in our project:
- extract_text_from_file() wraps PDF/DOCX reading in try/except and raise ValueError("friendly message") on failure (analyzer.py 941-966).
- analyze_multiple_resumes() wraps each resume in try/except so one bad file does not stop the whole batch (lines 874-895).
- In app.py, the Report tab wraps PDF creation in try/except to show a friendly error.

Why needed: Real users upload messy files. Exception handling keeps the app stable and the messages friendly.

2.11 Files and bytes (reading an uploaded file)

Definition: Computers store files as bytes (raw numbers). Text must be decoded from bytes using an encoding like UTF-8. A file-like object is anything you can call .read() on.

Analogy: Bytes are a sealed parcel; .decode("utf-8") is opening it to read the letter inside.

Tiny example:

```
raw = b"Hello"            # bytes (note the b)
text = raw.decode("utf-8") # 'Hello' (a normal string)
```

Where in our project: extract_text_from_file() (analyzer.py 903-970) reads the uploaded file's bytes with uploaded_file.read(), then decodes .txt directly, or uses pypdf / python-docx for .pdf / .docx.

Why needed: Uploaded resumes arrive as bytes; we must turn them into text before we can analyse them.

2.12 Classes and objects (just enough for the tests)

Definition: A class is a blueprint; an object is a thing built from it. A method is a function that belongs to the object.

Analogy: A class is the blueprint for a car; each actual car is an object.

Where in our project: We barely use classes in the main code (it is mostly simple functions - a deliberate, beginner-friendly choice). Classes appear in two places:
1. The tests: class TestKeywordMatch(unittest.TestCase): - each test is a method inside a class that inherits from unittest.TestCase.
2. The PDF report briefly uses FPDF() (an object from the fpdf2 library).

Why needed: unittest organises tests into classes. You only need to recognise the pattern, not write complex classes yourself.

2.13 Streamlit (turning Python into a web page)

Definition: Streamlit runs your Python script top-to-bottom and turns each st.something(...) call into a piece of a web page.

Key idea - "rerun": Every time you click a button or change a box, Streamlit runs the whole script again from the top. To remember things between reruns it uses st.session_state (a dictionary that survives reruns).

Analogy: Streamlit is like a whiteboard that gets wiped and redrawn every time you interact. st.session_state is a sticky note that stays on the board.

The Streamlit tools this project uses:

Call	What it draws / does
st.set_page_config(...)	Sets the browser tab title and layout (must be first).
st.sidebar.radio(...)	The left-hand menu to switch pages.
st.text_area(...)	The big box to paste a resume.
st.file_uploader(...)	The upload button for .pdf/.docx/.txt.

Call	What it draws / does
st.button(...)	A clickable button (e.g. "Analyze").
st.metric(...)	A big number card (e.g. "Match 100%").
st.tabs([...])	The result tabs (Overview, Skills, ...).
st.dataframe(...)	Shows a pandas table.
st.plotly_chart(...)	Shows a Plotly chart.
st.download_button(...)	Lets the user download the PDF/CSV.
st.session_state	Remembers the resume text and results across reruns.

Where in our project: all of app.py.

Why needed: It gives a clean, interactive web page using only Python - no need to learn HTML, CSS or JavaScript. (See Chapter 3 for a deeper look at the rerun/session_state rule, which caused and fixed a real bug in this project.)

2.14 pandas (tables of data)

Definition: pandas gives a DataFrame, which is a table (rows and columns) you can display and export.

Tiny example:

```
import pandas as pd
df = pd.DataFrame([{"Skill": "python", "In Resume": True}])
csv_text = df.to_csv(index=False)   # turn the table into CSV text
```

Where in our project: app.py builds DataFrames for the skill-gap table, the comparison table, and the batch results, and calls .to_csv(...) to make the downloadable CSV.

Why needed: Easy, neat tables and one-line CSV export.

2.15 Plotly (charts)

Definition: Plotly draws interactive charts (you can hover over them).

Where in our project: gauge_chart, matched_missing_bar, category_coverage_chart in app.py (lines 51-113) build Plotly figures that st.plotly_chart then displays.

Why needed: Charts make the match result and skill gaps easy to *see*, not just read.

2.16 Virtual environments & pip (setting up)

Definition: pip installs external packages. A virtual environment (venv) is a private box of packages for this project only, so it does not clash with other projects.

Analogy: A separate pencil case for each subject, so your maths pens don't mix with your art pens.

The commands (full details in Chapter 5):

```
python -m venv venv        # create the private box
venv\Scripts\activate      # (Windows) step into the box
pip install -r requirements.txt   # install the project's packages
```

Why needed: It guarantees the exact packages the project needs are present, without breaking anything else on the computer.

2.17 Checklist for this chapter

- [ ] I can explain a dictionary-of-lists using SKILL_DATABASE.
- [ ] I can compute intersection and difference by hand on two small sets.
- [ ] I know what \b does and why r won't match inside react.
- [ ] I can read a list comprehension out loud.
- [ ] I know why Streamlit "reruns" and what st.session_state is for.
- [ ] I can explain try/except with the uploaded-file example.

Next: Chapter 3 walks through every important file and function, using the real code.

Chapter 3 - File-by-File Code Explanation

> *How to read this chapter: We go through the real code, one file at a time, in dependency order (data first, then the engine that uses it, then the web page that uses the engine, then the tests). Each function says: its purpose, inputs, outputs, the important lines, and a small example. Line numbers refer to the real files.*
>
> *Runtime entry point reminder: when you *run* the app, Python starts at app.py (because you type streamlit run app.py). But app.py imports analyzer.py, which imports sample_data.py. So to *understand* the code it is easiest to read them in the reverse (dependency) order below.*

Files covered in this chapter:
1. sample_data.py (the data)
2. analyzer.py (the engine) - all 15 sections
3. app.py (the web page)
4. tests/test_analyzer.py (the tests)

3.1 sample_data.py - the data layer

Purpose: Hold all the *data* the app ships with, as embedded strings (text typed directly into the file). It contains no logic - nothing is calculated here.

What it defines:

(a) SKILL_DATABASE - a dictionary of lists

```
SKILL_DATABASE = {
    "Programming Languages": ["python", "java", "c++", "c#", "javascript", ...],
    "Web Development":       ["streamlit", "flask", "django", "node.js", ...],
    "Databases":             ["sql", "mysql", "postgresql", "mongodb", ...],
    ...
}
```

- Key = a category name (a string).
- Value = a list of skill words, all lowercase (so matching is case-insensitive).
- There are 10 categories. Multiword skills ("machine learning", "spring boot", "power bi") and special-character skills ("c++", "c#", ".net", "node.js", "ci/cd", "scikit-learn") are included on purpose to show off the matching logic.

Why lowercase? Because analyzer.py lowercases the resume text too, then compares. Same case on both sides = reliable matching.

(b) Five sample resumes

Each is a triple-quoted string (a string that can span many lines):

```
RESUME_PYTHON_DEVELOPER = """
John Mathew
Email: john.mathew@example.com | Phone: +91 98765 43210
...
Technical Skills
Python, SQL, Flask, Streamlit, pandas, NumPy, Git, Docker, REST APIs, Linux
...
```

```
"""
```

They are collected into a dictionary so the UI can list them in a dropdown:

```
SAMPLE_RESUMES = {
    "Python Developer": RESUME_PYTHON_DEVELOPER,
    "Data Analyst":     RESUME_DATA_ANALYST,
    "Java Developer":   RESUME_JAVA_DEVELOPER,
    "AI/ML Engineer":   RESUME_AIML_ENGINEER,
    "Cloud Engineer":   RESUME_CLOUD_ENGINEER,
}
```

(c) Five sample job descriptions

Same idea, collected into SAMPLE_JOB_DESCRIPTIONS with the same five keys.

(d) ROLE_SAMPLES - pairing a resume with its matching job

```
ROLE_SAMPLES = {
    role: {
        "resume": SAMPLE_RESUMES[role],
        "job_description": SAMPLE_JOB_DESCRIPTIONS[role],
    }
    for role in SAMPLE_RESUMES
}
```

This is a dictionary comprehension: for each role, store its resume and job together. The app's "Load by role" demo button uses this to fill both boxes at once.

How it connects: analyzer.py imports SKILL_DATABASE; app.py imports SAMPLE_RESUMES, SAMPLE_JOB_DESCRIPTIONS, ROLE_SAMPLES, and SKILL_DATABASE.

Beginner mistake to avoid: editing a skill to UPPERCASE here would break matching, because the engine compares against lowercase text.

3.2 analyzer.py - the engine (the most important file)

It has 15 numbered sections. We cover each. Remember: this file never imports Streamlit, so it can be tested on its own.

```
import io
import re
from sample_data import SKILL_DATABASE
```

- io -> used to treat uploaded file bytes as a readable stream.
- re -> regular expressions, used all over.
- SKILL_DATABASE -> the skill list, used as the default for matching.

Section 1 - preprocess_text(text) (lines 44-77)

Purpose: Clean raw text so matching is reliable.
Input: any text (or even None/non-text). Output: a cleaned, lowercased, single-spaced string.

```
if not text or not isinstance(text, str):
    return ""                                   # (1) safety for empty/bad input
text = text.lower()                             # (2) lowercase everything
text = re.sub(r"[^a-z0-9+#./\-\s]", " ", text)   # (3) keep letters/digits/tech symbols; rest -> space
text = re.sub(r"\s+", " ", text).strip()         # (4) squeeze all spaces to one; trim ends
return text
```

- (1) If the input is empty or not a string, return "" instead of crashing.

- (2) "Python" becomes "python".
- (3) The pattern [^a-z0-9+#./\-\s] means "any character that is not a lowercase letter, digit, whitespace, or one of + # . / -". Those become a space. Keeping + # . / - is the clever part: it protects c++, c#, .net, node.js, ci/cd.
- (4) Turns runs of spaces/newlines into a single space, then trims.

Example: preprocess_text("C++, Python!\n\nSQL") -> "c++ python sql".

Remove this and... every later step gets confused by capital letters and punctuation, so matching would badly under-count.

Section 2 - Skill matching helpers (lines 102-188)

This section solves the hardest problem: matching a skill as a whole word without wrong matches.

_build_skill_pattern(skill) (lines 106-129) - builds a regex for one skill and caches it (so each pattern is compiled only once):

```
if any(ch in _SPECIAL_SKILL_CHARS for ch in skill):     # has + # . / - ?
    boundary = r"[a-z0-9+#]"
    pattern = r"(?<!{b}){s}(?!{b})".format(b=boundary, s=escaped)  # look-arounds
else:
    pattern = r"\b{s}\b".format(s=escaped)              # normal word boundaries
```

- Normal skills (letters only, like python, r) use \b...\b. The \b (word boundary) is why r matches the language *R* but not the r inside react (because inside react, r is touching the letter e, so there is no boundary).
- Special skills (c++, c#, .net, ...) cannot use \b, because + and # are not "word" characters. Instead we say: "not touching a letter, digit, + or # on either side" using look-behind (?<!...) and look-ahead (?!...). This matches c++ but not c+++ or ac++.

skill_in_text(skill, cleaned_text) (lines 132-137): returns True/False by searching the compiled pattern in the already-cleaned text.

extract_skills(text, skill_database) (lines 140-167): the main one.

```
cleaned = preprocess_text(text)              # clean once
found = {}
for category, skills in skill_database.items():   # each category
    matches = []
    for skill in skills:                     # each skill in that category
        if skill_in_text(skill, cleaned) and skill not in matches:
            matches.append(skill)
    if matches:
        found[category] = matches            # only keep non-empty categories
return found
```

Output: a dictionary like {"Programming Languages": ["python"], ...}.

flatten_skills(skills_by_category) (lines 170-179): merges all the category lists into one set (no duplicates) - ready for set maths.

skill_category_lookup(...) (lines 182-188): builds the reverse map {skill: category}, used to label rows in the skill-gap table.

Example:

```
extract_skills("I use Python and C++")
# {'Programming Languages': ['python', 'c++']}
```

```
flatten_skills(_)        # {'python', 'c++'}
```

Section 3 - count_resume_statistics(text) (lines 195-236)

Purpose: basic numbers about the resume. Input: raw text. Output:
a dict with word_count, character_count, line_count, sentence_count,
average_word_length.
  - word_count = len(text.split()) -> words = pieces separated by whitespace.
  - character_count = len(text) -> every character, spaces included.
  - line_count -> counts only non-empty lines (via a list comprehension).
  - sentence_count = len(re.findall(r"[.!?]+", text)) -> counts groups of
. ! ?. If there is text but no such mark, it is set to 1.
  - average_word_length -> total letters / number of words, rounded to 2 dp; if
there are no words it is 0.0 (so there is no divide-by-zero).

Section 4 - detect_contact_information(text) (lines 246-304)

Purpose: find contact details by pattern. Uses compiled regexes:
  - Email: [a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,} -> "stuff @ stuff . 2+
letters".
  - Phone: a loose digit pattern, then it keeps only candidates with 10-15
digits (to avoid random long numbers).
  - LinkedIn / GitHub: look for linkedin.com/... / github.com/....
  - Portfolio: any other website-looking URL.

Output: a dict of {"found": bool, "count": int} style entries. It also
returns the raw emails list for internal use only - the UI never displays
personal details.

Honest note (say this in viva): this is pattern-based, so it can miss odd
formats or occasionally over-match. The code comments say so.

Section 5 - detect_resume_sections(text) (lines 312-353)

Purpose: guess which resume sections are present. SECTION_KEYWORDS maps a
canonical name (e.g. "Work Experience") to alternative headings (e.g.
"employment history", "work history"). For each section it checks whether
any heading variant appears (as a whole word). Output: detected
(name->bool), present (list), and missing (list - described as *possibly*
missing, since a resume might use an unusual heading).

Section 6 - Job keywords + calculate_keyword_match(...) (lines 360-411) *

This is the core scoring.

extract_job_keywords(jd) = the set of recognised skills in the job
description (reuses extract_skills + flatten_skills).

calculate_keyword_match(resume_text, job_description):

```
resume_skills = extract_job_keywords(resume_text)   # a set
job_skills    = extract_job_keywords(job_description) # a set

matched = resume_skills.intersection(job_skills)   # in BOTH
missing = job_skills.difference(resume_skills)     # job needs, resume lacks
extra   = resume_skills.difference(job_skills)     # bonus

total = len(job_skills)
```

```
if total > 0:
    match_percentage = round(len(matched) / total * 100, 2)
else:
    match_percentage = 0.0        # no keywords in JD -> 0%, no crash
```

Output dict: sorted matched/missing/extra lists, the counts, and match_percentage. The if total > 0 check is what prevents a divide-by-zero when the job description has no recognised skills.

Section 7 - detect_experience_education(text) (lines 425-458)

Finds approximate signals: years of experience ((\d+)\+?\s*years?), degree keywords (from _DEGREE_KEYWORDS), graduation years (1950-2099), and booleans for internship/projects/certifications. Honest note: approximate, never invents qualifications.

Section 8 - Quality checks (lines 465-530)

has_excessive_whitespace(text) -> True if there are many blank lines or long runs of spaces. resume_quality_checks(...) -> a list of {"label", "passed", "note"} items (email present? skills section present? reasonable length? etc.). These are heuristics, clearly labelled - a failing check is a hint to review, not a penalty.

Section 9 - calculate_readiness_score(...) (lines 549-622) *

Purpose: a broader 0-100 score. The weights are documented and sum to 100:

```
DEFAULT_WEIGHTS = {"keyword_coverage": 60, "sections": 20,
                   "contact": 10, "readability": 10}
```

Each component is turned into a 0-100 percentage, then: contribution = score_pct / 100 * weight, and the total is the sum, clamped to 0-100 (round(max(0.0, min(100.0, total)), 2)).

- keyword_coverage = the match percentage from Section 6.
- sections = fraction of 4 important sections present x 100.
- contact = 50 for an email + 50 for a phone.
- readability = starts at 100; -50 if very short (<80 words); -30 if cluttered whitespace.

Output: {"components": {...per-component numbers...}, "total": number} so the UI can show exactly how the total was built.

Section 10 - analyze_skill_gaps(resume_skills, job_skills) (lines 629-663)

Builds one row per required skill: skill, category, in_resume (bool), required_by_job (always True here), and an honest recommendation. If the skill is missing, the recommendation says *"if you genuinely have it, add it; otherwise learn it"* - it never tells anyone to lie.

Section 11 - Strengths & suggestions (lines 670-747)

generate_resume_strengths(analysis) -> evidence-backed positives (e.g. "Matched 5 of 7 keywords"). Every strength is tied to something actually detected.

generate_improvement_suggestions(analysis) -> if/else rules producing tips (no email? add one. missing keywords? review them honestly. too short? add detail.). Each tip is linked to a real detected condition, so tips never

contradict the analysis.

Section 12 - analyze_resume(...) (lines 754-809) * the conductor

Purpose: run the whole pipeline and return one big results dictionary. This is the function the UI and the compare/batch features all call, so the matching logic lives in exactly one place.

```
statistics   = count_resume_statistics(resume_text)
contact      = detect_contact_information(resume_text)
sections     = detect_resume_sections(resume_text)
exp_edu      = detect_experience_education(resume_text)
resume_skills = extract_skills(resume_text)
job_skills    = extract_skills(job_description)
...
keyword_match = calculate_keyword_match(resume_text, job_description)
skill_gap     = analyze_skill_gaps(resume_skills_flat, job_skills_flat)
readiness     = calculate_readiness_score(keyword_match, sections, contact, statistics, resume_text)
quality       = resume_quality_checks(...)
analysis = { ...all of the above... }
analysis["strengths"]   = generate_resume_strengths(analysis)
analysis["suggestions"] = generate_improvement_suggestions(analysis)
return analysis
```

Output: a dictionary with keys like statistics, contact, sections, keyword_match, skill_gap, readiness, strengths, suggestions, raw_text, job_title. Everything the UI needs is in this one object.

Section 13 - Compare & batch (lines 816-896)

compare_resumes(resume_a, resume_b, job_description, ...) -> runs analyze_resume on both and returns a tidy side-by-side summary list.

analyze_multiple_resumes(resumes, job_description, max_files=10) -> screens a list of resumes. Each item can be plain text or (name, text). Each is wrapped in try/except, so one broken file does not stop the batch (the failure is recorded in an error field). max_files caps how many are processed.

Section 14 - extract_text_from_file(uploaded_file) (lines 903-970)

Purpose: turn an uploaded .txt/.pdf/.docx into text.
- Reads the raw bytes with .read().
- .txt -> raw.decode("utf-8").
- .pdf -> uses pypdf's PdfReader, joins each page's extract_text(). If the result is empty it raises a friendly ValueError explaining it may be a scanned/image PDF (needs OCR, out of scope).
- .docx -> uses python-docx to join paragraph text.
- Any other extension -> a friendly ValueError.

Section 15 - Reports: create_analysis_report & create_analysis_pdf

create_analysis_report(analysis) (lines 977-1054) -> builds a Markdown string (used for the on-screen preview).

_pdf_safe(text) (lines 1057-1063) -> replaces characters the basic PDF font cannot draw, so PDF generation never crashes.

create_analysis_pdf(analysis) (lines 1066-1170) -> builds a PDF with fpdf2 and returns it as bytes for the download button. It mirrors the Markdown report so the preview and the PDF stay consistent. (Note the little

write() helper passes new_x="LMARGIN", new_y="NEXT" - this is required by fpdf2 so each line starts at the left margin; without it fpdf2 errors.)

3.3 app.py - the web page (the UI layer)

Purpose: everything the user sees and clicks. It imports the engine and the data:

```
import analyzer
from sample_data import (ROLE_SAMPLES, SAMPLE_JOB_DESCRIPTIONS,
                         SAMPLE_RESUMES, SKILL_DATABASE)
```

Page setup & disclaimer (lines 34-48)

st.set_page_config(...) sets the tab title, icon and wide layout. A DISCLAIMER string is shown on the main pages to keep the tool honest.

Chart helpers (lines 51-113)

- gauge_chart(value, title) -> a 0-100 dial (used for Match % and Readiness).
- matched_missing_bar(keyword_match) -> a green/red bar of matched vs missing.
- category_coverage_chart(resume_skills, job_skills) -> grouped bars comparing, per category, how many skills the resume has vs the job wants.

The session_state pattern (lines ~100-145) - important!

Because Streamlit reruns the whole script on every click, the text boxes store their content in st.session_state under keys like "resume_area" and "job_area". Small callback functions fill those boxes safely:

- _load_sample_resume() / _load_sample_job() / _load_role_demo() -> put sample text into the boxes before the widgets are drawn.
- _reset_all() -> clears the boxes and the stored analysis.

> *The bug this fixed (great viva story): originally the text area used both value=... and key="resume_area". Streamlit ignores value= once a widget has a key, so an uploaded resume appeared to extract successfully but the box stayed empty. The fix: stop using value=, and write directly to the widget's session_state key (via callbacks or before the widget is created).*

resume_input_block() (lines 180-223)

Draws the resume input: a radio to choose *Paste / Upload / Use a sample*. For uploads it calls analyzer.extract_text_from_file(...) and only extracts once per new file (tracked by name+size) so your edits are not wiped on every rerun. The editable text area (key="resume_area") is the single source of truth that gets analysed.

job_input_block() (lines 225-252)

Same idea for the job description, plus an optional job-title box and a "Load by role" demo button.

page_analyze() (lines 254-285)

The main page. It shows the two input blocks, an Analyze button and a Reset button. Validation: if the resume or job text is empty it shows an error and stops. Otherwise it calls analyzer.analyze_resume(...) and stores the result in st.session_state["analysis"], then calls render_dashboard.

render_dashboard(analysis) (lines 287-447)

Shows the results. Top row = four st.metric cards (Match %, Readiness, Matched, Missing). Then st.tabs([...]) creates seven tabs:
1. Overview - the two gauges, the matched/missing bar, the readiness breakdown table, statistics, contact categories, sections.
2. Skills - the category-coverage chart and the skill lists.
3. Skill Gap - the per-skill table + a Download CSV button.
4. Quality Checks - the heuristic checklist + approximate exp/education.
5. Strengths & Suggestions - two columns.
6. Raw vs Clean - shows the text before and after preprocess_text, so you can *see* preprocessing working.
7. Report - the Markdown preview + a Download PDF button (analyzer.create_analysis_pdf).

page_compare() (lines 449-489)

Pick two sample resumes + one job; calls analyzer.compare_resumes(...); shows a side-by-side table and a comparison bar chart.

page_batch() (lines 492-550)

Upload up to 10 resumes (or use the 5 samples); each is turned into (name, text) - the file name is used as the label - then analyzer.analyze_multiple_resumes(...) screens them. Shows a sorted table and a Download CSV button. The "Resume" column and the CSV use the real file names.

page_methodology() and page_about() (lines 553-614)

Static explanation pages (the algorithm; limitations and responsible use).

Sidebar navigation (lines 616-632)

```
PAGES = { " Home": page_home, " Analyze Resume": page_analyze, ... }
selection = st.sidebar.radio("Navigate", list(PAGES.keys()))
PAGES[selection]()     # call the chosen page's function
```

PAGES is a dictionary whose values are functions. The radio returns the chosen label; PAGES[selection]() then calls that page's function. This is a clean, simple router.

3.4 tests/test_analyzer.py - the tests

Purpose: automatically prove the engine is correct. Uses Python's built-in unittest. Tests are grouped into classes like TestKeywordMatch, TestSkillMatching, TestContactDetection, etc. Each method is one test.

Example test (the famous "R" case):

```
def test_single_letter_r_matches_only_as_word(self):
    self.assertTrue(analyzer.skill_in_text("r", analyzer.preprocess_text("skills: python, r, sql")))
    self.assertFalse(analyzer.skill_in_text("r", analyzer.preprocess_text("react and recruiting")))
```

- assertTrue(...) -> the test passes only if the value is True.
- assertFalse(...) -> passes only if False.
- So this proves r matches the standalone language but not inside react.

There are 31 tests in total, covering good matches, no matches, empty inputs, empty job descriptions (no divide-by-zero), mixed case, c++/c#/.net/node.js, the r case, multiple emails, alternate headings, duplicate skills,

whitespace, comparison ordering, batch robustness, and PDF generation. How to run them is in Chapter 5.

3.5 Files-explained checklist

- [x] sample_data.py - fully explained.
- [x] analyzer.py - all 15 sections explained.
- [x] app.py - all pages and the session_state pattern explained.
- [x] tests/test_analyzer.py - purpose and structure explained.
- [x] requirements.txt, README.md, PROJECT_REPORT.md, VIVA_GUIDE.md, .gitignore - described in Chapter 1 (no complex logic to walk through).

Nothing important remains unexplained. The only files not walked through line-by-line are plain documentation and the auto-generated __pycache__ / venv folders, which contain no project logic.

Next: Chapter 4 traces exactly what happens when you click Analyze, and explains the key algorithms with worked examples.

Chapter 4 - Execution Flow & Algorithms

How to read this chapter: First we follow the program from the moment you run it, step by step. Then we zoom into the four most important algorithms and work through them with real numbers. Diagrams are plain text so they print cleanly.

4.1 How the program starts (startup sequence)

1. You type streamlit run app.py in the terminal.
2. Streamlit starts a small local web server and runs app.py top to bottom.
3. As app.py loads, its import analyzer line runs analyzer.py, whose from sample_data import SKILL_DATABASE line runs sample_data.py. So all three files are now loaded into memory.
4. st.set_page_config(...) sets the page title/layout (this must be the first Streamlit command).
5. The sidebar is drawn: selection = st.sidebar.radio("Navigate", ...).
6. PAGES[selection]() calls the function for the chosen page (default: Home).
7. Your browser opens at http://localhost:8501 and shows the page.

Key Streamlit idea again: every click or text change makes Streamlit re-run app.py from the top. Values that must survive a rerun live in st.session_state.

4.2 What happens when you click "Analyze" (end-to-end trace)

Imagine you loaded the Python Developer sample and clicked Analyze.

```
USER clicks "Analyze"
      |
      v
app.py: page_analyze()
      |  reads resume text from st.session_state["resume_area"]
      |  reads job text   from st.session_state["job_area"]
      |  VALIDATION: if either is empty -> show st.error() and STOP
      v
app.py calls analyzer.analyze_resume(resume_text, job_text, job_title)
      |
      v
analyzer.analyze_resume()  (the conductor)
      +- count_resume_statistics(resume_text)      -> words, lines, ...
      +- detect_contact_information(resume_text)   -> email/phone/...
      +- detect_resume_sections(resume_text)       -> which sections
      +- detect_experience_education(resume_text)  -> approx signals
      +- extract_skills(resume_text)   -- each calls preprocess_text()
      +- extract_skills(job_text)      -- then _build_skill_pattern() per skill
      +- calculate_keyword_match(resume_text, job_text)
      |        + set intersection / difference  -> matched / missing / match %
      +- analyze_skill_gaps(resume_skills, job_skills)
      +- calculate_readiness_score(...)            -> weighted 0..100
      +- resume_quality_checks(...)
      +- generate_resume_strengths(analysis)
      +- generate_improvement_suggestions(analysis)
```

```
      |
      v  returns ONE big dictionary `analysis`
app.py stores it in st.session_state["analysis"]
      |
      v
app.py: render_dashboard(analysis)
      +- st.metric() x4  (Match %, Readiness, Matched, Missing)
      +- st.tabs([...])  (Overview, Skills, Skill Gap, Quality,
                          Strengths, Raw vs Clean, Report)
      |
      v
USER sees numbers, tables, charts; can download PDF / CSV
```

Error path: if a step inside the engine raised an error it would bubble up; for file uploads and batch screening the code deliberately catches errors with try/except and shows a friendly message instead of crashing.

4.3 What each major button does

Where	Button	What happens in the code
Analyze page	Load this sample resume	callback _load_sample_resume() writes the chosen sample into session_state["resume_area"].
Analyze page	Load resume + job for this role	callback _load_role_demo() fills both boxes from ROLE_SAMPLES.
Analyze page	Analyze	validates inputs, calls analyze_resume(), renders the dashboard.
Analyze page	Clear / Reset	callback _reset_all() empties the boxes and the stored analysis.
Report tab	Download PDF	analyzer.create_analysis_pdf(analysis) returns bytes to st.download_button.
Skill Gap tab	Download CSV	pandas DataFrame.to_csv(...) feeds st.download_button.
Compare page	Compare	analyzer.compare_resumes(a, b, jd) -> table + bar chart.
Batch page	Screen resumes	extracts each file's text, calls analyze_multiple_resumes(...), shows a sorted table + CSV.

4.4 Algorithm 1 - Text preprocessing

What it does: turns messy raw text into clean, lowercase, single-spaced text while protecting tech symbols.

Pseudocode:

```
function preprocess_text(text):
    if text is empty or not a string: return ""
    text = lowercase(text)
    replace every char that is NOT [a-z 0-9 space + # . / -] with a space
    collapse multiple spaces into one; trim the ends
    return text
```

Worked example:

```
Input : "C++, Python!!!   Node.js\n\nSQL"
step 1 : already handled (string)
step 2 : "c++, python!!!   node.js\n\nsql"
```

```
step 3 : "c++  python     node.js  sql"   (commas, ! -> spaces; + # . / - kept)
step 4 : "c++ python node.js sql"
Output : "c++ python node.js sql"
```

Complexity: O(n) in the length of the text (each character is processed a constant number of times). Edge cases handled: empty input, non-string input, text with lots of punctuation or blank lines.

4.5 Algorithm 2 - Safe skill matching (the regex trick)

Problem: we must match a skill as a whole token. Two traps:
- r must match the language *R*, but not the r in react.
- c++ must match c++, but \b (word boundary) does not work because + is not a word character.

Pseudocode:

```
function build_pattern(skill):
    if skill contains any of + # . / - :
        boundary = one of [a-z 0-9 + #]
        pattern  = (not boundary) + skill + (not boundary)   # look-arounds
    else:
        pattern  = \b + skill + \b                            # word boundaries
    return compiled, case-insensitive pattern
```

Why it works:

Text	Skill	Pattern used	Match?	Reason
python, r, sql	r	\br\b	[x] yes	r has spaces/commas around it = boundaries
react	r	\br\b	[ ] no	r touches e, so no boundary after it
c++ developer	c++	(?<![a-z0-9+#])c\+\+(?![a-z0-9+#])	[x] yes	nothing letter/+/# touches it
c+++	c++	same	[ ] no	a + follows, which is forbidden
node.json	node.js	look-arounds	[ ] no	an o follows node.js, forbidden

Caching: every pattern is compiled once and stored in _skill_pattern_cache, so repeated matching is fast. Complexity: matching one skill in text is roughly O(n) in the text length; extracting all skills is O(S x n) where S is the number of skills - small and fast in practice.

Edge case / limitation: .net won't match inside asp.net (the p touches it). This is a documented trade-off that avoids worse false matches.

4.6 Algorithm 3 - Keyword match score (set maths) *

What it does: compares resume skills with job skills and produces a %.

Pseudocode:

```
resume_skills = set of skills found in resume
job_skills    = set of skills found in job description
matched = resume_skills  intersect  job_skills      # intersection (in both)
missing = job_skills - resume_skills       # difference (job needs, resume lacks)
if size(job_skills) == 0:
```

```
    match% = 0                              # avoid divide-by-zero
else:
    match% = round(size(matched) / size(job_skills) * 100, 2)
```

Worked example:

```
resume_skills = {python, sql, flask, git}
job_skills    = {python, sql, flask, docker, aws}

matched = {python, sql, flask}        -> size 3
missing = {docker, aws}               -> size 2
total job skills = 5
match% = 3 / 5 * 100 = 60.0%
```

Why a set and not a list? Sets drop duplicates automatically and give us `intersect` and `-` directly, which is both fast and easy to explain. Complexity: set intersection/difference are about O(k) where k is the number of skills - very fast.

Edge cases: empty resume -> matched empty -> 0%. Empty/keyword-free job description -> total = 0 -> the `if` returns 0% with no crash.

4.7 Algorithm 4 - Readiness score (weighted blend)

What it does: combines four signals into one 0-100 score. The weights are fixed and sum to 100.

The formula (plain language):

```
readiness =  0.60 x (keyword match %)
          +  0.20 x (percent of 4 key sections present)
          +  0.10 x (contact completeness: 50 for email + 50 for phone)
          +  0.10 x (readability: 100, minus 50 if <80 words, minus 30 if cluttered)
then clamp the result to the range 0..100
```

Variables explained:

- *keyword match %* - from Algorithm 3 (0-100).
- *sections* - of [Work Experience, Education, Technical Skills, Projects], how many are present, as a percentage.
- *contact* - 0, 50, or 100.
- *readability* - starts at 100 and loses points for being too short or cluttered.

Worked example (a strong resume):

```
keyword match % = 100     -> contributes 0.60 x 100 = 60
all 4 sections present    -> 100%  -> 0.20 x 100 = 20
email + phone found       -> 100   -> 0.10 x 100 = 10
long, tidy resume         -> 100   -> 0.10 x 100 = 10
TOTAL = 60 + 20 + 10 + 10 = 100  (clamped to 100)
```

Worked example (a weaker resume):

```
keyword match % = 40      -> 0.60 x 40 = 24
2 of 4 sections (50%)     -> 0.20 x 50 = 10
email only (50)           -> 0.10 x 50 = 5
short resume (100-50=50)  -> 0.10 x 50 = 5
TOTAL = 24 + 10 + 5 + 5 = 44
```

Why clamp? So the score can never be below 0 or above 100, even if weights were changed. Edge case: empty inputs give 0 across the board -> total 0, no crash. The UI shows each component's contribution so nothing is a "black box".

4.8 Putting it together - a mini sequence diagram

```
app.py            analyzer.py                          sample_data.py
  |                   |                                     |
  |-- analyze_resume->|                                     |
  |                   |-- extract_skills(resume) ---------->| (reads SKILL_DATABASE)
  |                   |<------- skills by category ---------|
  |                   |-- extract_skills(job) ------------->|
  |                   |<------- skills by category ---------|
  |                   |-- calculate_keyword_match (sets)    |
  |                   |-- calculate_readiness_score         |
  |                   |-- generate strengths/suggestions    |
  |<--- analysis dict-|                                     |
  |-- render_dashboard (metrics, tabs, charts)              |
```

4.9 Checklist for this chapter

- [ ] I can list the startup steps from streamlit run app.py to the browser.
- [ ] I can trace what happens when "Analyze" is clicked.
- [ ] I can work out a match % from two small skill sets by hand.
- [ ] I can explain why \b handles r but not c++.
- [ ] I can compute a readiness score from the four components.

Next: Chapter 5 shows how to install, run, test, and troubleshoot the project.

Chapter 5 - Installation, Testing & Troubleshooting

How to read this chapter: Follow it top to bottom the first time. It gets the project running, shows how to test it, and lists fixes for common problems. Commands are given for Windows and macOS/Linux.

5.1 Prerequisites

You need	Why	How to check
Python 3.11 or newer	to run the code	python --version
pip	to install packages (comes with Python)	pip --version
A web browser	to view the app	already on your computer
(Recommended) VS Code	a friendly code editor	see the project's main README.md

Verified: this project was developed and its tests were run successfully on Python 3.14.6 on Windows. Any Python 3.11+ should work.

5.2 Step-by-step setup

Open a terminal inside the project folder (Resume Analyzer/).

Windows (PowerShell)

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

macOS / Linux

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

What each command does:
1. python -m venv venv -> creates a private package box named venv/.
2. activate -> steps into that box (your prompt shows (venv)).
3. pip install -r requirements.txt -> installs the 6 packages the project needs (streamlit, pandas, plotly, pypdf, python-docx, fpdf2).
4. streamlit run app.py -> starts the app.

5.3 What you should see when it works

- The terminal prints something like:

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
```

- Your browser opens automatically to that address and shows ResumeIQ with a sidebar on the left.
- If the browser does not open by itself, open it and visit

http://localhost:8501 manually.

> *Honesty note: these are the *expected* messages. Always confirm on your own machine - do not assume success without seeing the app in the browser.*

5.4 Configuration & environment variables

There are none. This project has no API keys, no passwords, no database connection strings, and no .env file. Nothing secret is stored or needed.
That is a genuine strength: there is nothing to leak and nothing to configure.
Everything it needs (sample data, the skill list) is embedded in sample_data.py.

5.5 Trying the main feature (manual test)

1. In the sidebar, click Analyze Resume.
2. For the job description method, choose Load by role and pick Python Developer, then click Load resume + job for this role.
3. Click Analyze.
4. You should see Keyword Match 100% and a high Readiness Score.
5. Open the Raw vs Clean tab to watch preprocessing in action.
6. Open the Report tab and click Download PDF.

To see a lower, different score (good for demos): load the Data Analyst sample resume but keep the Python Developer job description, then Analyze - the score drops because skills like flask, streamlit, docker are missing.

5.6 Running the automated tests

From the project folder (with the virtual environment active):

```
python -m unittest discover -s tests
```

Expected output (last lines):

```
Ran 31 tests in 0.8s
OK
```

> *Verified result: when these documents were prepared, the command above was run and reported Ran 31 tests ... OK (all 31 passed) on Python 3.14.6.*
> *Re-run it on your machine and capture a screenshot for your report.*

Run a single test class (handy when demonstrating one idea):

```
python -m unittest tests.test_analyzer.TestSkillMatching
```

What the tests check (grouped)

Test group	Proves
TestPreprocessing	lowercasing, trimming, keeping c++/node.js, handling None.
TestSkillMatching	c++/c#/.net/node.js match; r matches only as a word; duplicates counted once.
TestKeywordMatch	good match >50%; no-match = 0%; empty resume/JD safe; missing list correct.
TestContactDetection	multiple emails; missing contact; phone found.
TestSections	alternate headings map to canonical sections.
TestStatistics	empty text safe; blank lines ignored; whitespace flag.

Test group	Proves
TestReadinessScore	score always 0-100; weights sum to 100; empty inputs safe.
TestComparisonAndBatch	comparison ordering; batch survives a bad item; max_files respected.
TestReport	Markdown report has the scores; PDF starts with %PDF.

5.7 Stopping the app

- Click the terminal, then press Ctrl + C (Windows/Linux) or Control + C (macOS). The server stops.
- To leave the virtual environment later, type deactivate.

5.8 Troubleshooting (common errors & fixes)

Symptom	Likely cause	Fix
python is not recognized (Windows)	Python not on PATH	Reinstall Python and tick "Add python.exe to PATH".
pip is not recognized	virtual env not active	Run venv\Scripts\activate (Win) or source venv/bin/activate (Mac) first.
streamlit: command not found	packages not installed	Re-run pip install -r requirements.txt with the env active.
ModuleNotFoundError: No module named 'fpdf'	fpdf2 missing	pip install fpdf2 (or reinstall requirements).
Uploaded PDF box is empty / "No selectable text"	scanned/image-only PDF	Use a .docx or .txt, or a text-based PDF (OCR is out of scope).
Port 8501 already in use	another app is running on it	streamlit run app.py --server.port 8502 then open localhost:8502.
Charts do not appear	Plotly not installed or stale browser	reinstall requirements; refresh the page.
App shows "Please provide resume text"	input box empty	paste/upload/load a sample before clicking Analyze.

5.9 How to debug (for a beginner)

1. Read the red error box in the app or the traceback in the terminal - the last line names the error type and message.
2. Find the file and line number in the traceback; open that spot.
3. Add a temporary print(...) just before the failing line to see the values (e.g. print("resume words:", len(resume_text.split()))).
4. Re-run and read the printed value - it usually reveals the problem (often an empty or unexpected input).
5. Remove the print once fixed.
6. For logic questions, run the tests - a failing test points straight at the function that misbehaves.

5.10 Edge cases that matter (and are handled)

- Empty resume or empty job description -> scores become 0, no crash.
- Job description with no recognised skills -> total = 0, match % = 0 (the if total > 0 guard prevents divide-by-zero).
- Tricky skills c++, c#, .net, node.js, ci/cd -> matched correctly.

- Single-letter skill r -> not matched inside react/year.
- A broken file in a batch -> caught; that row shows an error, the rest continue.

5.11 Known limitations (be honest in the viva)

- Keyword matching, not meaning: misses synonyms/paraphrases (false negatives).
- Cannot judge real skill level or seniority.
- No ATS guarantee.
- Contact/section detection is pattern-based and can misfire.
- No OCR: scanned/image PDFs are not supported.
- Privacy: files are processed in memory and not stored, but the batch table (by your request) shows the uploaded file names, which may contain personal names - mention this if asked.

5.12 Checklist for this chapter

- [ ] I can set up and run the app from scratch on my OS.
- [ ] I know there are no secrets/config to worry about.
- [ ] I can run the tests and read the OK / failure output.
- [ ] I can stop the app and deactivate the environment.
- [ ] I can fix the three most common errors without help.

Next: Chapter 6 is full viva preparation - explanations at three lengths, a demo script, and 50+ questions with answers.

Chapter 6 - College Viva Preparation

*How to use this chapter: Read the three explanations aloud until they feel natural (don't memorise word-for-word - understand them). Practise the demo script on the running app. Then go through the questions and make sure you can *explain* each answer, not just recite it.*

6.1 Explain the project in three lengths

A. The 30-second version

"ResumeIQ is a Python and Streamlit web app that checks how well a resume matches a job description. It cleans both texts, looks for known skills in each, and uses set maths - the overlap - to give a match score and a list of missing skills. It's fully rule-based and offline, so every number can be explained."

B. The 2-minute version

"The problem I'm solving is that comparing a resume to a job by hand is slow and inconsistent. ResumeIQ automates the first-pass keyword screen.

It has three layers. sample_data.py holds the data - a skill dictionary and some sample resumes and jobs, all as embedded strings. analyzer.py is the engine: it cleans text with preprocess_text, finds skills with regular expressions, and compares them. app.py is the Streamlit web page with the buttons, charts, and downloads.

The core is set theory. I put the resume's skills in one set and the job's skills in another. The intersection is the matched skills; the difference (job minus resume) is the missing skills. The match percentage is matched divided by total job skills, times 100. I also build a readiness score out of four weighted parts - keywords 60%, sections 20%, contact 10%, readability 10% - which always add up to 100.

It's tested with 31 automated unittest cases, and it's honest about its limits: it matches keywords, it doesn't understand meaning."

C. The 5-minute version (add these points to B)

- Why Streamlit, not Flask: Streamlit builds an interactive data app with charts in pure Python - no HTML/JavaScript - which suits a single-developer academic project.
- The clever matching: \b word boundaries stop r matching inside react; special look-arounds handle c++, c#, .net, node.js, ci/cd which \b can't. Patterns are compiled once and cached.
- The conductor function: analyze_resume() runs every step and returns one dictionary, so the UI, the compare feature, and the batch feature all reuse the same logic - no duplicated matching code.
- Extra features: upload .pdf/.docx/.txt (via pypdf / python-docx), compare two resumes, batch-screen many, download a PDF report (fpdf2) and CSV exports.
- Safety & honesty: no divide-by-zero (guarded), scores clamped 0-100, files processed in memory, no secrets, no AI over-claims.

- A real engineering moment: I hit and fixed a Streamlit session_state bug where a text area with both value= and a key= ignored the uploaded text; the fix was to use callbacks and write to the widget's session-state key.

6.2 Demonstration script (step-by-step)

#	Do this	Say this
1	streamlit run app.py; show the Home page	"This runs entirely offline on my machine."
2	Open Methodology, expand the skill database	"These embedded strings are my skill vocabulary."
3	On Analyze, choose Load by role -> Python Developer	"Embedded samples let it work instantly."
4	Click Analyze; show the metric cards	"Match % and readiness, computed by rules."
5	Open Raw vs Clean	"Here's preprocessing: lowercased and cleaned, but c++/node.js survive."
6	Open Skills; show matched vs missing	"Matched is the set intersection; missing is the set difference."
7	Open Overview gauges; expand readiness breakdown	"The four weights add up to 100 - nothing hidden."
8	Open Skill Gap; download CSV	"One honest recommendation per required skill."
9	Open Report; download PDF	"A shareable PDF report."
10	Switch to Data Analyst resume vs the same Python job	"A lower score, because skills are missing - the tool is discriminating, not random."
11	Open About & Limitations	"It matches words, not meaning - a human must stay in the loop."
12	In the terminal, run the tests	"Ran 31 tests ... OK - the logic is verified."

6.3 Fifty+ viva questions with answers

Objectives & problem statement

1. What does your project do? It compares a resume to a job description and reports a keyword match score, missing skills, and improvement tips.
2. What problem does it solve? It automates the slow, inconsistent first-pass keyword screening of resumes and gives transparent feedback.
3. Who is it for? Students/job seekers, recruiters (first pass), and faculty evaluating the project.
4. Why is it "academically defensible"? Every number comes from a documented formula; the code is simple, tested, and honest about limits.
5. Is it AI? No. It is rule-based string processing - no machine learning, no external AI service.

Embedded strings & data

6. What are embedded strings here? Sample resumes, job descriptions, and the skill list written directly into sample_data.py using triple quotes.
7. Why triple-quoted strings? They let a whole multi-line resume be one string literal, keeping line breaks.
8. What is SKILL_DATABASE? A dictionary whose keys are categories and whose values are lists of skill words - a "dictionary of lists".
9. Why store skills in lowercase? Because the engine lowercases the resume too, so matching is case-insensitive and reliable.

10. Why no database? The data is tiny and static, and user resumes are not stored (for privacy), so a database would add complexity with no benefit.

Architecture & files

11. Describe the architecture. Three layers: app.py (UI), analyzer.py (logic), sample_data.py (data), plus tests/.
12. Why separate the layers? Separation of concerns - each is simple, testable, and replaceable on its own.
13. Which file is the entry point? app.py, run via streamlit run app.py.
14. Why can the tests run without Streamlit? Because analyzer.py never imports Streamlit; the tests import only the engine.
15. What does analyze_resume() do? It runs the whole pipeline and returns one results dictionary used by the UI, compare, and batch features.

Python concepts

16. Difference between a list and a set? A list is ordered and allows duplicates; a set is unordered, unique, and supports intersection/difference.
17. Why use sets for matching? They remove duplicates and give the overlap and difference directly and fast.
18. What is intersection? Items present in both sets - the matched skills.
19. What is difference? Items in the first set but not the second - e.g. job - resume = missing skills.
20. What is a dictionary? A collection of key: value pairs; used for SKILL_DATABASE and the big results dictionary.
21. What is a list comprehension? A one-line way to build a list with a loop, e.g. counting non-empty lines.
22. What is an f-string? A string with {variables} inside, prefixed with f, used to build messages and report lines.
23. What is truthiness? Empty values ("", [], 0, None) act as False in an if; used to guard empty inputs.

Regular expressions & matching

24. What is a regular expression? A pattern language for finding shapes in text; Python's re module runs them.
25. What does \b do? Matches a word boundary (the edge of a word), so \br\b matches a standalone r.
26. Why doesn't r match in react? Because inside react the r touches e, so there is no word boundary after it.
27. How do you match c++? With look-arounds (?<![a-z0-9+#])c\+\+(?![a-z0-9+#]), because \b doesn't work with +.
28. Why cache compiled patterns? So each skill's regex is built once, making repeated matching faster.
29. How are multiword skills like "machine learning" matched? After whitespace is collapsed to single spaces, the exact phrase is matched with \b.
30. What is a false positive / false negative? False positive = flagging something that isn't really there; false negative = missing something that is there.

Scoring

31. How is the keyword match score calculated? matched / total_job_keywords x 100.
32. What if the job description has no keywords? total = 0, so the code returns 0% without dividing by zero.
33. What are the readiness weights? Keywords 60, sections 20, contact 10, readability 10 - they sum to 100.
34. How do you stop the score going above 100 or below 0? It is clamped with max(0, min(100, total)).
35. Why show each component's contribution? For transparency - the user sees exactly how the total was built.

Text processing & statistics

36. What does preprocess_text do? Lowercases, replaces noise punctuation with spaces (keeping + # . / -), and collapses whitespace.
37. Why keep + # . /? To protect tech names like c++, c#, .net, node.js, ci/cd.
38. How do you count words? len(text.split()).
39. How do you count sentences? Count groups of . ! ? with a regex (set to 1 if there is text but no terminator).
40. How is average word length computed, safely? Total letters / word count; if there are no words it returns 0.0 to avoid divide-by-zero.

Files, libraries, testing

41. How are PDF/DOCX read? pypdf extracts PDF page text; python-docx reads Word paragraphs; .txt is decoded directly.
42. Why can scanned PDFs fail? They are images with no selectable text; extraction returns nothing, so a clear error is raised (OCR is out of scope).
43. Why pandas? To show neat tables and export CSV with one line.
44. Why Plotly? For interactive gauges and bar charts.
45. Why fpdf2? To create the downloadable PDF report in pure Python, no system dependencies.
46. How is the project tested? 31 unittest cases in tests/test_analyzer .py, run with python -m unittest discover -s tests.
47. Did the tests pass? Yes - Ran 31 tests ... OK (verified on Python 3.14.6).
48. How is personal data protected? Resumes are processed in memory, not stored; contact details are detected but not displayed in summaries.

Limitations & future work

49. Biggest limitation? It matches keywords, not meaning, so synonyms are missed.
50. Can it guarantee ATS compatibility? No - it is a heuristic indicator, not an ATS.
51. How would you improve it? Add semantic matching (embeddings/NLP models), OCR for scanned PDFs, learned skill weights, and optional storage - all listed as *future* work, not implemented.
52. Why keep it rule-based for now? So it stays fully explainable, offline, and suitable for a mini-project viva.

6.4 Code-specific questions (based on the real files)

1. Why does extract_skills take skill_database=SKILL_DATABASE as a default parameter? So you can call it with just text in normal use, but still pass a different skill set in tests.
2. In calculate_keyword_match, what does job_skills.difference(resume_ skills) contain? The missing skills - those the job needs that the resume lacks.
3. What happens in preprocess_text("")? The if not text guard returns "" immediately.
4. What would break if you removed the if total > 0 check in calculate_keyword_match? A job description with no skills would cause a divide-by-zero crash.
5. What does the try/except in analyze_multiple_resumes protect against? One unreadable/odd resume crashing the entire batch; the error is recorded per row instead.
6. Why is resume_area used as the text area's key? So its content survives Streamlit reruns and is the single source of truth for analysis.
7. What does PAGES[selection]() do? Looks up the chosen page's function in the PAGES dictionary and calls it.
8. What does flatten_skills return and why a set? One set of all skills; a set removes duplicates and enables set maths.
9. If you changed DEFAULT_WEIGHTS to {"keyword_coverage": 100} only, what happens? Readiness would equal the keyword match %; the code still clamps 0-100. (You'd want the weights to sum to 100 to keep it meaningful.)
10. Why does create_analysis_pdf return bytes? Because st.download_button needs bytes (or text) as the file data.

6.5 Explain it to different audiences

- To a 12-year-old (30s): "It reads your CV and a job advert, and tells you which important words match and which are missing - like a spell-checker, but for job skills."
- To a classmate (2 min): use section 6.1.B.
- To an examiner (5 min): use section 6.1.C, then offer to show the code for calculate_keyword_match and run the tests.

Next: Chapter 7 - glossary, a one-page revision sheet, and a final self-assessment checklist.

Chapter 7 - Glossary & Revision Notes

How to use this chapter: Use the glossary to look up any word quickly. Use the one-page revision sheet the night before your viva. Finish with the self-assessment checklist - if you can tick every box, you are ready.

7.1 Glossary (plain definitions + project example)

Term	Simple meaning	In this project
Module	A Python file you can import.	analyzer.py, sample_data.py.
Import	Borrow code from another file.	from sample_data import SKILL_DATABASE.
Function	Named block of code with inputs/output.	preprocess_text(text).
Parameter / argument	Input to a function.	text in preprocess_text.
Default parameter	Optional input with a preset value.	skill_database=SKILL_DATABASE.
Return value	What a function hands back.	analyze_resume returns the results dict.
String	Text data.	the resume and job text.
Triple-quoted string	A multi-line string in """ """.	each sample resume.
List	Ordered collection, allows duplicates.	a category's skills.
Dictionary	key: value pairs.	SKILL_DATABASE, the results dict.
Dictionary of lists	Dict whose values are lists.	SKILL_DATABASE.
Set	Unordered, unique collection.	resume skills, job skills.
Intersection	Items in both sets.	matched skills.
Difference	Items in one set but not the other.	missing skills (job - resume).
Comprehension	One-line loop to build a list/dict.	counting non-empty lines.
f-string	Text with {variables} inside.	report/message lines.
Truthiness	Empty values act as False.	if not text.
Regular expression (regex)	Pattern language for text.	finding emails, skills.
re	Python's regex module.	used across analyzer.py.
Word boundary \b	The edge of a word.	matching r safely.
Look-around	"(not) followed/preceded by X".	matching c++.
Compile / cache (regex)	Build a pattern once, reuse it.	_skill_pattern_cache.
Exception	A runtime error.	bad file upload.
try/except	Catch an error and continue.	file reading, batch.
raise	Deliberately signal an error.	raise ValueError(...).
Bytes	Raw file data (not yet text).	uploaded file contents.
Decode	Turn bytes into text.	.decode("utf-8").
Class	A blueprint for objects.	unittest.TestCase, FPDF.
unittest	Python's built-in testing tool.	tests/test_analyzer.py.
assertion	A test's pass/fail check.	assertEqual, assertTrue.
Streamlit	Python -> web page framework.	all of app.py.
Widget	A UI control (button, text box).	st.button, st.text_area.
Rerun	Streamlit re-runs the script on each action.	why session_state is needed.
session_state	Streamlit's memory across reruns.	stores resume text + results.
Callback	Function run when a widget changes.	_load_sample_resume.
DataFrame	A pandas table.	skill-gap / batch tables.
CSV	Comma-separated values file.	table downloads.
Plotly figure	An interactive chart object.	gauges, bar charts.
ATS	Applicant Tracking System (recruiter software).	what we *cannot* guarantee.
OCR	Reading text from images.	needed for scanned PDFs (out of scope).
Heuristic	A rule-of-thumb, not a guarantee.	quality checks, scores.
Virtual environment (venv)	Per-project package box.	created during setup.
pip	Python's package installer.	installs requirements.

7.2 One-page revision sheet

Project: ResumeIQ - Smart Resume Analyzer (Python + Streamlit).
Goal: compare a resume to a job description; report match %, missing skills, tips. Rule-based, offline, no AI, no database, no secrets.

Three layers:
- app.py -> UI (Streamlit): pages, inputs, charts, downloads.
- analyzer.py -> engine: cleaning, matching, scoring, reports.
- sample_data.py -> data: SKILL_DATABASE + 5 resumes + 5 jobs (embedded).

Core maths (memorise):

```
matched = resume_skills  intersect  job_skills
missing = job_skills - resume_skills
match % = matched / total_job_skills x 100   (0 if no job skills)
readiness = 0.6xkeywords + 0.2xsections + 0.1xcontact + 0.1xreadability  (0-100)
```

Matching trick: \b for normal skills (r react); look-arounds for c++, c#, .net, node.js, ci/cd.

Conductor: analyze_resume() runs everything and returns one dict.

Tested: 31 unittest cases -> Ran 31 tests ... OK (Python 3.14.6).

Run: streamlit run app.py · Test: python -m unittest discover -s tests.

Honest limits: keywords not meaning; no proficiency judgement; no ATS guarantee; no OCR.

7.3 The 20 most important things to remember

1. It matches keywords, it does not understand meaning.
2. "Embedded strings" = text in the code, not AI embeddings.
3. Three layers: UI (app.py), logic (analyzer.py), data (sample_data.py).
4. SKILL_DATABASE is a dictionary of lists, all lowercase.
5. Matching uses sets: intersection = matched, difference = missing.
6. Match % = matched / total job skills x 100.
7. The if total > 0 check prevents divide-by-zero.
8. Readiness weights (60/20/10/10) sum to 100 and the total is clamped.
9. \b makes r match only as a whole word.
10. c++/c#/.net/node.js need look-arounds, not \b.
11. preprocess_text keeps + # . / - to protect tech names.
12. analyze_resume() is the single conductor reused everywhere.
13. app.py is the entry point; streamlit run app.py starts it.
14. Streamlit reruns the script on every action; session_state remembers.
15. The uploaded-text bug was caused by value= + key= on a widget.
16. Files (pdf/docx/txt) are read by pypdf / python-docx / direct decode.
17. Scanned PDFs have no text -> handled with a clear error (no OCR).
18. 31 unittest cases verify the engine; they don't need the web page.
19. No database, no API keys, no secrets - nothing to leak or configure.
20. It never guesses sensitive attributes and keeps a human in the loop.

7.4 Important files and their responsibilities

File	Responsibility
app.py	Streamlit UI: pages, inputs, validation, charts, downloads.
analyzer.py	Engine: preprocessing, matching, scoring, reports.
sample_data.py	Embedded data: skills, sample resumes, sample jobs.
tests/test_analyzer.py	31 automated tests for the engine.
requirements.txt	The 6 packages to install.
README.md	Beginner install & run guide.
PROJECT_REPORT.md	Formal academic report.

7.5 Most important functions and their purpose

Function (in analyzer.py)	Purpose
preprocess_text(text)	Clean + lowercase text, keep tech symbols.
_build_skill_pattern(skill)	Build/cache a safe regex per skill.
extract_skills(text, db)	Find skills by category in text.
flatten_skills(...)	Merge category lists into one set.
count_resume_statistics(text)	Words, chars, lines, sentences, avg length.
detect_contact_information(text)	Find email/phone/LinkedIn/GitHub/portfolio.
detect_resume_sections(text)	Detect sections (with alternate headings).
calculate_keyword_match(r, j)	Set maths -> matched/missing/extra/match %.
calculate_readiness_score(...)	Weighted 0-100 score with breakdown.
analyze_skill_gaps(r, j)	Per-skill table with honest recommendations.
generate_resume_strengths(a)	Evidence-backed positives.
generate_improvement_suggestions(a)	Rule-based, honest tips.
analyze_resume(...)	Conductor: runs all steps, returns one dict.
compare_resumes(...)	Two resumes vs one job, side by side.
analyze_multiple_resumes(...)	Batch screening, robust to bad files.
extract_text_from_file(f)	Read .txt/.pdf/.docx into text.
create_analysis_report(a)	Markdown report string.
create_analysis_pdf(a)	PDF report bytes (fpdf2).

7.6 Execution flow in one line

```
streamlit run app.py -> choose page -> input resume+job -> Analyze ->
analyze_resume() -> {preprocess -> extract_skills -> set match -> readiness ->
strengths/suggestions} -> render_dashboard() -> metrics/charts/tabs -> download PDF/CSV
```

7.7 Common viva mistakes to avoid

- [ ] Saying "it uses AI/ML". [x] It is rule-based; no ML.
- [ ] Confusing embedded strings with AI embeddings.
- [ ] Saying it "understands" the resume. [x] It matches keywords.
- [ ] Forgetting the divide-by-zero guard when asked about empty inputs.
- [ ] Claiming it guarantees ATS success. [x] It's a heuristic indicator.
- [ ] Saying \b handles c++ (it doesn't - look-arounds do).
- [ ] Not being able to compute a match % by hand. Practise it!
- [ ] Saying the tests need the web page. [x] They test analyzer.py directly.

7.8 Final self-assessment checklist

- [ ] I can explain the project in 30 seconds, 2 minutes, and 5 minutes.
- [ ] I can draw the three-layer architecture.
- [ ] I can compute matched/missing/match % from two small sets.
- [ ] I can compute a readiness score from the four components.
- [ ] I can explain \b vs look-arounds with the r and c++ examples.
- [ ] I can name what analyze_resume() returns.
- [ ] I can run the app and the tests, and read the results.
- [ ] I can state three honest limitations.
- [ ] I can explain the Streamlit rerun / session_state idea and the bug it fixed.
- [ ] I can answer any question in Chapter 6 in my own words.

If every box is ticked, you are ready for your viva. Good luck!